SOKOINE UNIVERSITY

SUA

OF AGRICULTURE

ABE 715: Computer Programming

Compendium Part 5

Advanced Applications: AI, IoT, and GIS

Prepared for:
MSc. Irrigation Engineering and Management
MSc. Agricultural and Biosystems Engineering

Instructor: Dr. Kadeghe G. Fue
Electronics and Precision Agriculture Lab

Department of Agricultural Engineering
Sokoine University of Agriculture (SUA)

Academic Year 2025/2026

Contents

Preface ii

1 Machine Learning in Agriculture 1
- 1.1 The Paradigm Shift . . . 1
- 1.2 Types of Machine Learning . . . 1
- 1.3 Supervised Learning: Regression . . . 1
  - 1.3.1 Python Implementation: Scikit-Learn . . . 2
- 1.4 Supervised Learning: Classification . . . 2
- 1.5 Evaluation Metrics . . . 3

2 Internet of Things (IoT) Integration 4
- 2.1 The IoT Architecture . . . 4
- 2.2 Hardware: The ESP32 . . . 4
- 2.3 Communication: MQTT Protocol . . . 4
- 2.4 Implementation Code . . . 5
  - 2.4.1 The Edge (C++ on ESP32) . . . 5
  - 2.4.2 The Server (Python Listener) . . . 5

3 Geospatial Data Analysis (GIS) 6
- 3.1 Spatial Intelligence . . . 6
- 3.2 Data Types . . . 6
- 3.3 Coordinate Reference Systems (CRS) . . . 6
- 3.4 GeoPandas: Python for Vector Data . . . 6
- 3.5 Remote Sensing: NDVI . . . 7

4 Future Trends and Course Conclusion 8
- 4.1 The Digital Twin . . . 8
- 4.2 Edge Computing . . . 8
- 4.3 Course Conclusion . . . 8

Comprehensive Assignment 5 9

Preface to Part 5

The final module of ABE 715 represents the convergence of software and the physical world. We move beyond processing static data on a laptop to interacting with live environments.

Machine Learning (ML) allows us to move from explicit programming (where we write the rules) to implicit learning (where the computer finds the rules from data). This is crucial for complex biological systems where formulas like Manning's Equation do not exist (e.g., predicting pest outbreaks).

Internet of Things (IoT) connects our software to the field. We learn to program microcontrollers (ESP32) to sense the environment and transmit data to the cloud via MQTT protocols.

Geospatial Analysis (GIS) gives our data context. In agriculture, *where* something happens is as important as *what* happened. We utilize Python's powerful geospatial stack to analyze satellite imagery and vector maps.

Learning Objectives for Part 5:

- Train and evaluate Machine Learning models (Regression and Classification) for crop yield and disease detection.
- Architect a complete IoT system from sensor to server using MQTT.
- Perform spatial operations (CRS projection, NDVI calculation) using GeoPandas and Rasterio.
- Synthesize all course concepts into a cohesive understanding of Digital Twins and Smart Farming.

Chapter 1

Machine Learning in Agriculture

1.1 The Paradigm Shift

In traditional programming (Lectures 1-12), we input **Data** and **Rules** to get **Answers**. In Machine Learning, we input **Data** and **Answers** (Labels) to get **Rules**.

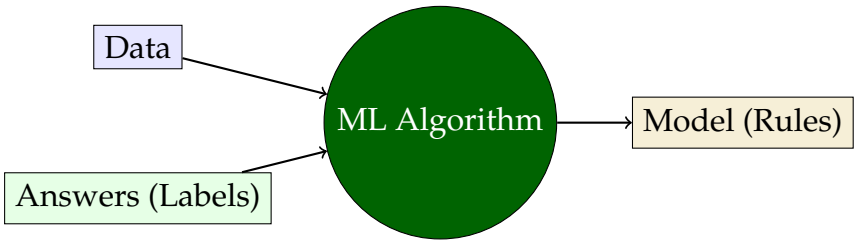


Figure 1.1: The Machine Learning Workflow

1.2 Types of Machine Learning

- **Supervised Learning:** The most common in AgTech. We teach the computer with labeled examples.
  - *Regression:* Predicting a continuous number (e.g., Crop Yield in tons/ha).
  - *Classification:* Predicting a category (e.g., Disease: Yes/No).
- **Unsupervised Learning:** The computer finds hidden patterns in unlabeled data.
  - *Clustering:* Grouping soil samples into management zones.

1.3 Supervised Learning: Regression

We use **Linear Regression** to model the relationship between variables. Formula: $y = mx + c$ (or $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 ...$)

1.3.1 Python Implementation: Scikit-Learn

We use the `sklearn` library, the industry standard for classical ML.

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Training Data
# X = [Rainfall (mm), Fertilizer (kg/ha)]
X_train = np.array([[100, 50], [120, 60], [80, 40], [150,
    80]])
# y = Yield (tons/ha)
y_train = np.array([2.5, 3.0, 2.0, 4.5])

# Initialize and Train
model = LinearRegression()
model.fit(X_train, y_train)

# Predict for a new season
# Rain=130mm, Fertilizer=70kg
X_new = np.array([[130, 70]])
prediction = model.predict(X_new)

print(f"Predicted Yield: {prediction[0]:.2f} tons/ha")
print(f"Coefficients: {model.coef_}")
```

Listing 1.1: Yield Prediction Model

1.4 Supervised Learning: Classification

For categorical problems, we often use **K-Nearest Neighbors (KNN)**. It classifies a new data point based on the majority vote of its closest neighbors in the feature space.

```
from sklearn.neighbors import KNeighborsClassifier

# X = [pH, Nitrogen]
X = [[5.5, 20], [6.0, 25], [7.5, 10], [8.0, 5]]
# y = Class Labels (0=Tea, 1=Beans)
y = [0, 0, 1, 1]

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X, y)

# Predict for pH 5.8, N 22
result = knn.predict([[5.8, 22]])
crop = "Tea" if result[0] == 0 else "Beans"
print(f"Recommended Crop: {crop}")
```

Listing 1.2: Crop Classification

1.5 Evaluation Metrics

How do we know if our model is good?

- **Regression:** Mean Squared Error (MSE) and R^2 Score (Accuracy of fit).
- **Classification:** Accuracy (%) and Confusion Matrix (True Positives vs False Positives).

Chapter 2

Internet of Things (IoT) Integration

2.1 The IoT Architecture

IoT is not just about sensors; it is about the flow of data.

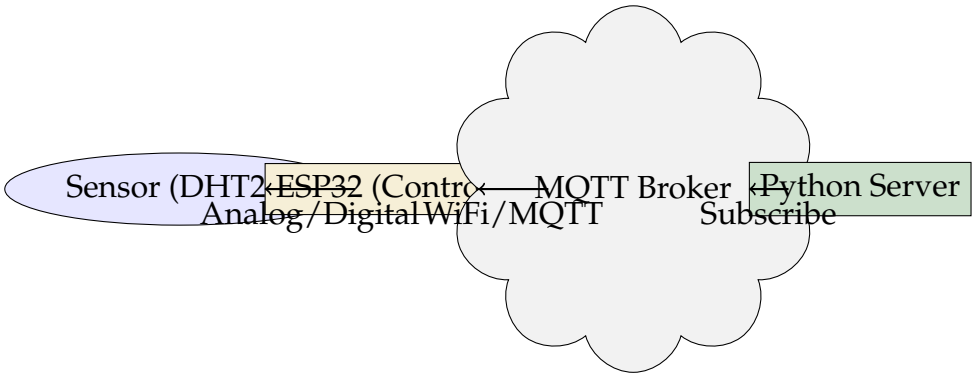


Figure 2.1: Data Flow in an Agricultural IoT System

2.2 Hardware: The ESP32

We prefer the ESP32 microcontroller because:

1. Integrated WiFi and Bluetooth.
2. Dual-core processor (can handle networking and sensing simultaneously).
3. Multiple ADC (Analog-to-Digital Converter) channels for soil sensors.
4. Low cost (< $10 USD).

2.3 Communication: MQTT Protocol

HTTP (used for websites) is too heavy for small sensors. We use **MQTT** (Message Queuing Telemetry Transport).

- **Publish:** The sensor sends data to a specific "Topic" (e.g., `farm/blockA/temp`).

- **Subscribe:** The server listens to that topic.
- **Broker:** The middleman managing the traffic (e.g., Mosquitto, HiveMQ).

2.4 Implementation Code

2.4.1 The Edge (C++ on ESP32)

```cpp
#include <WiFi.h>
#include <PubSubClient.h>

void loop() {
  float hum = dht.readHumidity();
  float temp = dht.readTemperature();

  // Create JSON string
  String payload = "{\"temp\":";
  payload += temp;
  payload += "}";

  // Publish to Broker
  client.publish("sua/farm/sensors", (char*) payload.c_str())
   ;

  delay(5000); // Wait 5 seconds
}
```

Listing 2.1: ESP32 Publishing Data

2.4.2 The Server (Python Listener)

```python
import paho.mqtt.client as mqtt
import json

def on_message(client, userdata, msg):
    payload = msg.payload.decode()
    data = json.loads(payload)
    print(f"Received Temp: {data['temp']} C")

    # Simple Logic
    if data['temp'] > 30:
        print("ALERT: High Temperature!")

client = mqtt.Client()
client.connect("broker.hivemq.com", 1883)
client.subscribe("sua/farm/sensors")
client.on_message = on_message
client.loop_forever()
```

Listing 2.2: Python MQTT Listener

Chapter 3

Geospatial Data Analysis (GIS)

3.1 Spatial Intelligence

Knowing *what* the problem is (e.g., Low Yield) is useless if you don't know *where* it is. GIS allows us to map data to physical locations.

3.2 Data Types

1. **Vector:** Discrete geometric shapes.
    - **Points:** Wells, Trees, Sensor Locations.
    - **Lines:** Rivers, Pipes, Roads.
    - **Polygons:** Field boundaries, Lakes, Buildings.
2. **Raster:** Continuous grids (pixels).
    - Satellite Imagery, Elevation Models, Soil pH maps.

3.3 Coordinate Reference Systems (CRS)

The earth is round; maps are flat. The CRS defines how we flatten the earth.

- **EPSG:4326 (WGS84):** Uses Latitude/Longitude (Degrees). Good for GPS location.
- **Projected CRS (e.g., UTM):** Uses Meters. Essential for calculating Areas and Distances accurately.

3.4 GeoPandas: Python for Vector Data

GeoPandas extends standard Pandas to handle spatial operations.

```
import geopandas as gpd

# Load Shapefile
gdf = gpd.read_file("farm_plots.shp")
```

```
5
6 # Reproject to UTM Zone 37S (Meters)
7 gdf = gdf.to_crs(epsg=32737)
8
9 # Calculate Area
10 gdf["area_ha"] = gdf.geometry.area / 10000
11
12 print(gdf[["plot_id", "area_ha"]])
```

Listing 3.1: Calculating Field Area

3.5 Remote Sensing: NDVI

The **Normalized Difference Vegetation Index (NDVI)** measures plant health based on how they reflect light. Healthy plants reflect Near-Infrared (NIR) light and absorb Red light.

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

We calculate this using **Rasterio** and NumPy arrays on satellite images. Values range from -1 to 1. Values > 0.5 indicate healthy dense vegetation.

Chapter 4

Future Trends and Course Conclusion

4.1 The Digital Twin

The ultimate goal of agricultural computing is the **Digital Twin**: a virtual replica of the physical farm.

- Real-time data updates the model state (IoT).
- AI predicts future states (Machine Learning).
- Simulation tests management decisions (ODEs).

4.2 Edge Computing

Sending high-definition video from a drone to the cloud is slow and expensive. **Edge Computing** involves running AI models *on the device* (e.g., on the tractor's computer) to make decisions in milliseconds (e.g., See Weed $\rightarrow$ Open Nozzle).

4.3 Course Conclusion

You have journeyed from printing "Hello World" to designing complex systems that integrate data science, hardware, and spatial analysis. As Agricultural Engineers, your code is as powerful as your plow. Use these tools to solve the pressing challenges of food security and resource efficiency.

Comprehensive Assignment 5 (Modules 13-16)

Instructions

Submit solutions for the following scenarios. This assignment tests your ability to synthesize advanced concepts.

Question 1: Yield Prediction (Machine Learning)

You have a dataset of historical yields. `X = [[rain, temp, fertilizer], ...] y = [yield, ...]` **Task:**

1. Create a dummy dataset with at least 10 rows.
2. Split the data (80% train, 20% test).
3. Train a **Linear Regression** model.
4. Predict the yield for a season with: 150mm Rain, 28°C Temp, 50kg Fertilizer.
5. Print the model's coefficients.

Question 2: IoT Data Simulator

Write a Python script that simulates a smart greenhouse sensor. **Task:**

1. Create a function `get_temp()` that generates a random temperature between 20 and 35.
2. Create a loop that runs 10 times (simulating 10 readings).
3. Inside the loop:
   - Get the temperature.
   - Formulate a JSON string: `{"sensor_id": 1, "temp": 24.5}`.
   - Print "Publishing to MQTT: [JSON String]".
   - If Temp $>$ 30, Print "ACTION: Fan ON".

Question 3: Spatial Filtering (GIS)

You have a list of GPS coordinates representing soil samples. `samples = [(-6.8, 37.6, 5.5), (-6.9, 37.7, 6.2), ...]` (Lat, Lon, pH). **Task:**

1. Create a GeoDataFrame from this data.
2. Filter the data to find samples where pH < 6.0 (Acidic).
3. Print the coordinates of the acidic soil samples requiring Lime application.

Question 4: Computer Vision Logic (Fruit Grading)

You are designing the logic for a fruit sorting camera. The camera returns: `diameter_mm` and `color_hsv_hue`. **Rules:**

- **Tomato:** Hue between 0-20.
- **Orange:** Hue between 20-40.
- **Grade A:** Diameter $>$ 70mm.
- **Grade B:** Diameter $50 - 70$mm.

Task: Write a function `grade_fruit(diameter, hue)` that returns a string like "Grade A Tomato" or "Grade B Orange". Test it with (75mm, 10) and (60mm, 30).

Question 5: The "Smart Farm" Pitch

Write a short (150-200 word) conceptual proposal for a Final Project that uses at least THREE concepts from this course (e.g., IoT + Python Logic + Database). Describe the problem, the solution, and the technologies used.